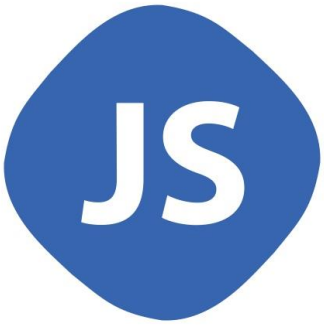


Метод addEventListener

- Введение
- Создание обработчика
- Удаление обработчика
- Не забываем про this
- Передача параметров в addEventListener
- Практика применения

Введение

Для взаимодействия с пользователем в JavaScript определен механизм событий. **Событие** – это сигнал браузера о том, что что-то произошло. Все DOM-узлы умеют подавать такие сигналы (хотя события бывают и не только в DOM). Чаще всего инициатором событий является **пользователь**, запросивший страницу.

Событию можно назначить обработчик – некоторый **программный код**, который сработает, как только событие **произошло**. Именно благодаря обработчикам JavaScript-код может реагировать на действия пользователя.

Есть **три способа** назначения **обработчиков событий**:

- **Атрибут** HTML.
- **DOM**-свойство.
- Специальный метод **addEventListener**().

Метод addEventListener()

Метод **addEventListener** - это самый функциональный способ позволяющий добавить обработчик события к указанному элементу и запустить выполнение программы при совершении заданного действия. Получить информацию о сигналах браузера можно из Document (DOM), Element, Window и других объектов поддерживающих события.



addEventListener() является одних из трёх способом прослушивать события, наряду с добавление **атрибута** к тегам в HTML и обращения к **свойствам** объекта напрямую.

Синтаксис

```
element.addEventListener(eventType, handler, options);
```

- **element** - объект, действие по которому отслеживаем.
- **eventType** - тип события, которое мы хотим отследить: клик, прокрутка мыши, нажатие клавиши и т.д. Написание чувствительно к регистру: click правильно, Click или CLICK неправильно.
- **handler**:
  - **функция JavaScript** (имя функции или сама функция), которая будет выполнена после наступления события.
  - **объект JavaScript**, который получает уведомление при возникновении события указанного типа. Это должен быть объект с **handleEvent**() методом.

> ***Важно***. Используя в качестве **обработчика** JS-объект с методом **handleEvent**() мы добьемся продвинутого использования метода **addEventListener**().

- **options** - необязательный объект со свойствами, внутри которого доступны следующие параметры (подробнее о параметрах в следующем уроке):
  - **capture**: значение записывается в формате **true** или **false** и задает этап, на котором будет обработано событие. По умолчанию **false** - на этапе всплытия, **true** - на этапе погружения (перехват).
  - **once**: значение типа **boolean**, по умолчанию **false**, если **true** тогда обработчик будет удален после выполнения.
  - **passive**: по умолчанию **false**, если **true** обработчик никогда не вызовет **preventDefault**(), взамен этого будет сгенерировано предупреждение.

Метод **addEventListener**() имеет важное преимущество перед остальными способами, метод позволяет навесить **несколько обработчиков на одно событие**. Это происходит из-за того, что у объекта только одно свойство, например с именем onclick (клик) или mousemove (движение мыши) и если обратиться к одному из них напрямую несколько раз, второй обработчик перезапишет первый.

> ***Примечание***. Также отследить некоторые события можно только с помощью **addEventListener**(), например навесить **обработчик** на **событие DOMContentLoaded** по другому не получится.

Метод removeEventListener()

Удалить обработчик события можно также с помощью метода **removeEventListener**(). В этом случае появляется возможность сделать это когда нам удобно. Для того, чтобы все сработало, первые два параметра у обоих методов должны быть идентичными.

> ***Важно***. Важно передать в **removeEventListener**() именно название функции, с анонимными функциями ничего не получиться, даже если записать код точь в точь.

Популярные события

Рассмотрим DOM-события, которые отслеживаются чаще других:

- **click** - клик левой кнопкой мыши по элементу, на сенсорных устройствах это касание;
- **contextmenu** - клик на элемент правой кнопкой мыши - вызов контекстного меню;
- **mouseover** / **mouseout** - наведение на элемент курсора мыши / курсор покидает элемент;
- **mousemove** - движение мыши;

- **keydown** / **keyup** - клавиша нажата / клавиша отпущена;
- **DOMContentLoaded** - весь HTML загружен, а DOM-дерево построено.

На один элемент можно назначить (навесить) сколько угодно событий (ну, если с логикой проблемы, почему нет ;). Ну или столько событий, сколько обеспечат выполнение поставленной задачи. Каким образом можно отследить эти события, разберем после выполнения демо-примера **example_1.**

Создание обработчика

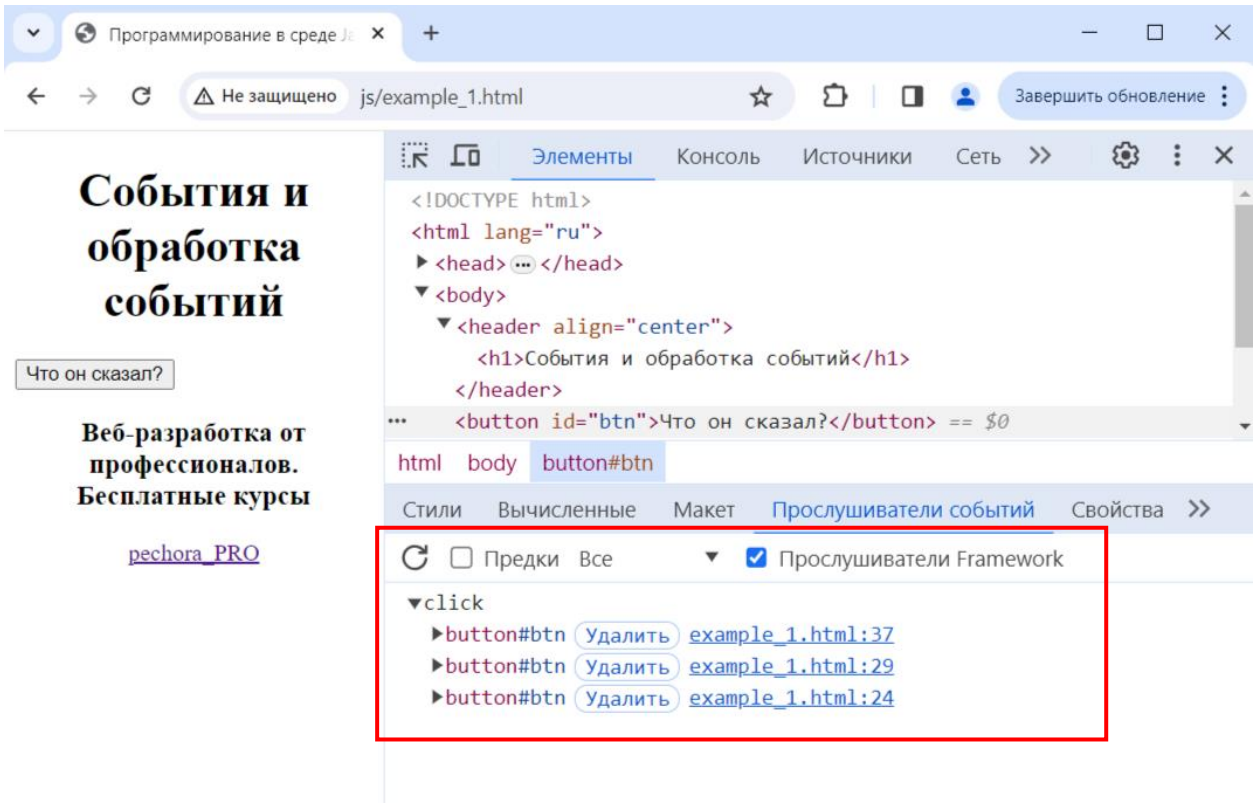
Вспомним, как навесить несколько обработчиков на один HTML-элемент.

example_1.

```
<button id=»btn»>Что он сказал?</button>
<script>
    // цитата
    let citate = undefined;
    // ссылка на элемент button
    const btn = document.getElementById(“btn”);
    // навешиваем на событие click элемента button несколько обработчиков
    // первый обработчик определяет глобальную переменную citate
    btn.addEventListener(«click», function() {
        citate = 'Умные не столько ищут одиночества, сколько избегают создаваемой дураками суеты (Артур Шопенгауэр).';
    });
    // второй обработчик выводит и переопределяет переменную citate
    btn.addEventListener(“click”, () => {
```

```
        alert(citate);
        citate = 'Ого, ну ты умный...';
    });
    // третий обработчик выводит новое значение переменной citate
    btn.addEventListener("click", fnOutCitate);
    function fnOutCitate() {
        alert(citate)
    }
</script>
```

На один элемент мы навесили три события **click**. Просмотреть назначенные события в **Google Chrome** можно следующим образом.



Некоторые браузеры позволяют просмотреть список событий, некоторые требуют установки дополнительных плагинов (не знаю, мне хрома хватает...).

Рассмотрим, как можно навесить обработчик на элемент из коллекции.

example_2.

```
<button class="btn">Контекст окна Window</button><p>
<button class="btn">Контекст элемента button</button>
<script>
    // контекст - объект window
    fnMessage0 = () => {
        alert(this);
    }
    // контекст - элемент button
    function fnMessage1 () {
        alert(this);
    }
    // получаем коллекцию элементов button
    const btns = document.querySelectorAll(".btn");
    // к элементам коллекции обращаемся по индексу
    btns[0].addEventListener("click", fnMessage0);
    btns[1].addEventListener("click", fnMessage1);
</script>
```

Всегда обращайте внимание на контекст **this**. В последних уроках я постоянно обращаюсь к вопросу контекста. Не станет исключением и этот урок.

Не забываем про this

В демонстрационном примере **example_3** обработчики, назначенные через метод **addEventListener** (как и обработчики, назначенные через **свойство**) в качестве **this** возвращают объект кнопки **button**.

Обратите внимание, обработчик – **функция-инструкция.**

example_3.

```
<button class="btn" id="btn1">Нажмите, пожалуйста 1</button><p>
<button class="btn" id="btn2">Нажмите, пожалуйста 2</button><p>
<button class="but" id="btn3">Нажмите, пожалуйста 3</button><p>
<button class="but" id="btn4">Нажмите, пожалуйста 4</button><p>
<script>
    // получаем коллекцию элементов button.btn
    let btns = document.getElementsByClassName("btn");
    // на каждый элемент коллекции навешиваем событие click и его
    обработчик
    for (btn of btns) {
        // используем метод addEventListener
        btn.addEventListener("click", function() {
            alert(this.innerText + ` ( id = ${this.id} )`);
            console.log(this);
        })
    }
    // получаем коллекцию элементов button.but
    let buts = document.getElementsByClassName("but");
    // на каждый элемент коллекции навешиваем событие click и его
    обработчик
    for (but of buts) {
        // используем свойство узла
        but.onclick = function() {
            alert(this.innerText + ` ( id = ${this.id} )`);
```

```
            console.log(this);
        }
    }
</script>
```

В такой же ситуации обработчики в виде **функции-стрелки** ведут себя иначе.

example_4.

```
<button class="btn" id="btn1">Нажмите, пожалуйста 1</button><p>
<button class="btn" id="btn2">Нажмите, пожалуйста 2</button><p>
<button class="but" id="btn3">Нажмите, пожалуйста 3</button><p>
<button class="but" id="btn4">Нажмите, пожалуйста 4</button><p>
<script>
    // получаем коллекцию элементов button.btn
    let btns = document.getElementsByClassName("btn");
    // на каждый элемент коллекции навешиваем событие click и его
    обработчик
    for (btn of btns) {
        // используем метод addEventListener
        btn.addEventListener("click", () => {
            alert(this.innerText + ` ( id = ${this.id} )`);
            console.log(this);
        })
    }
    // получаем коллекцию элементов button.but
    let buts = document.getElementsByClassName("but");
    // на каждый элемент коллекции навешиваем событие click и его
    обработчик
```

```
    for (but of buts) {
        // используем свойство узла
        but.onclick = () => {
            alert(this.innerText + ` ( id = ${this.id} )`);
            console.log(this);
        }
    }
</script>
```

Примечание. Даже если вы хорошо разобрались в поведении коварного **this**, не факт, что вы вспомните об этом через некоторое время. Не беда, достаточно помнить о его коварстве и при необходимости **проверять его содержимое**.

Впрочем, это предупреждение верно для многих конструкций всех языков программирования.

Примечание. Все возможные варианты поведения контекста **this** в обработчиках рассматривались в предыдущем уроке.

Удаление обработчика

Обработчик можно добавить, а можно и удалить. Для удаления нужно передать именно ту функцию-обработчик, которая была назначена.

Такой способ не сработает:

```
elem.addEventListener( "click", () => alert('Привет Мир!'));
// какой-то код...
```

```
elem.removeEventListener( "click", () => alert('Привет Мир!'));
```

Обработчик не будет удалён, т.к. в **removeEventListener**() передана не та же функция, а другая, с одинаковым кодом.

Правильный способ удаления:

```
function sayHello() {
    alert( 'Привет Мир!' );
}
elem.addEventListener( "click", sayHello);
// какой-то код...
elem.removeEventListener( "click", sayHello);
```

Обратите внимание, если функцию обработчик не сохранить где-либо, ее невозможно будет удалить. Нет метода, который позволит извлечь из целевого элемента обработчики событий, назначенные через **addEventListener**.

example_5.

```
<input type="checkbox"
onchange='fnMoodSwitch(this)'>Переключатель настроений<p>
<button id="btn">Нажмите, пожалуйста</button>
<script>
    btn.addEventListener("click", fnHandler1);
    // управление обработчиком события click
    function fnMoodSwitch(_this) {
        if (_this.checked) {
            btn.removeEventListener("click", fnHandler1);
            btn.addEventListener("click", fnHandler2);
        } else {
            btn.removeEventListener("click", fnHandler2);
```

```
            btn.addEventListener("click", fnHandler1);
        }
    }
    // объявим функции-обработчики
    // функция инструкция
    function fnHandler1() {
        // режим - пессимист
        alert("Жизнь такая, что хуже не бывает! (говорит
        пессимист)");
    }
    // функция стрелка
    fnHandler2 = () => {
        // режим - оптимист
        alert("Бывает, бывает... (говорит оптимист)");
    }
</script>
```

Передача параметров

Передать параметр инлайн функции при назначении обработчика в HTML-атрибуте достаточно тривиально.

```
<button id="btn" onclick="sayHello('Евгений')">Отправить
отзыв</button>
<script>
    function sayHello (name) {
        alert("Привет, " + name + '!');
    }
```

```
</script>
```

Разберемся, каким образом передать параметр при назначении обработчика в **addEventListener**.

```
elem.addEventListener("click", sayHello);
```

На самом деле есть несколько способов.

Метод bind()

Первый способ – хорошо известный и исправно работающий метод привязки контекста – метод **bind**().

example_6.

```
<button id="btn">Метод bind в привязке параметров</button>
<script>
    // функция трех аргументов
    function loadData(arg1, arg2, arg3) {
        alert(arg1);
        alert(arg2);
        alert(arg3);
    }
    // привязка контента с помощью метода bind
    btn.addEventListener("click", loadData.bind(null, 'Im first
    argument', 'Hello, User', 'Надеюсь понятно объяснил?'));
</script>
```

Усложним пример **привязкой** реальной переменной – массива данных.

example_7.

```
<!-- контейнер для вставки контента -->
```

```
<select id="albums"></select>
<script>
    let albums = [
    [1, "Atom Heart Mother", "10 октября 1970", "EMI, Harvest,
    Capitol", "LP, CD", "Золотой (USA)"],
    [2, "Meddle", "30 октября 1971", "EMI, Harvest, Capitol",
    "Vinyl, кассета, CD", "Платиновый (USA)"],
    /* ... */
    ];
    // загрузке DOM назначаем функцию-обработчик loadData
    // функции привязываем массив данных
    document.addEventListener("DOMContentLoaded",
    loadData.bind(null, albums));
    // определение функции-обработчика
    function loadData(arr) {
        // перебираем элементы массива
        arr.forEach( elem => {
        document.getElementById("albums").insertAdjacentHTML("bef
        oreend", `<option id="${elem[0]}">${elem[1]}</option>`);
        } )
    }
</script>
```

Примечание. Мы подошли вплотную к событию **event**. В следующем уроке разберем что это, и как можно использовать.

Привязка контекста с помощью метода **bind**() – весьма хороший способ. Но используется довольно редко. Все дело в том, что есть **объект-обработчик.** Вот в нем вся сила, брат ;)

Метод handleEvent() объекта-обработчика

Еще раз напомню, **обработчиком события** в методе **addEventListener**() может быть:

- **функция JavaScript** (имя функции или сама функция), которая будет выполнена после наступления события.
- **объект JavaScript**, который получает уведомление при возникновении события указанного типа. Это должен быть объект с **handleEvent**() методом.

А теперь примеры.

example_8.

```
<button id="btn">Тест свойства объекта-обработчика</button>
<script>
    // null-объект в качестве обработчика
    // метод handleEvent содержит ссылку на функцию sayHello
    btn.addEventListener("click", {handleEvent: sayHello, msg:
    "Hello User!" });
    // функция с параметром null-объекта
    function sayHello() {
        alert(this.msg); // msg - свойство null-объекта
    }
</script>
```

Определение метода может быть расположено в самом объекте-обработчике.

```
// другой вариант написания метода
btn.addEventListener("click", { handleEvent() { alert(this.msg)
}, msg: "Hello User!" });
```

Первый пример передачи данных в объект-обработчик простой.

example_9.

```
<h2>Дискография группы Pink Floyd</h2>
<table border=1>
    <tr>
        <td>1</td>
        <td>Atom Heart Mother</td>
        <td>10 октября 1970</td>
        <td>EMI, Harvest, Capitol</td>
        <td>LP, CD</td>
        <td>Золотой (USA)</td>
    </tr>
    <!-- /// -->
</table>
<script>
    // получаем коллекцию строк
    let trs = document.querySelectorAll('table tr');
    // навесим обработчик на событие click строк
    trs.forEach( (tr, key) => {
        tr.addEventListener("click", {
            name:tr.children.item(1).innerText, // свойство null-
            объекта
            handleEvent() {  // метод null-объекта
```

```
            // при клике по строке выведем название альбома
            alert(this.name)
        }
    })
})
</script>
```

И еще один пример. Теперь немного сложнее.

example_10.

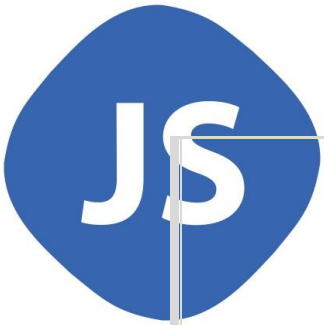
```
<button class="citate">Цитата Великого кормчего</button><p>
<button class="citate">Цитата Великого кормчего</button><p>
<button class="citate">Цитата Великого кормчего</button><p>
<!-- файл цитат -->
<script src="citateMao.js"></script>
<script>
    // коллекция элементов button
    btns = document.querySelectorAll(".citate");
    // на каждый элемент button навесим событие
    btns.forEach( (btn) => {
        // количество цитат
        let max = citate.length;
        // случайное целое число в заданном интервале
        let num = Math.floor(Math.random() * max);
        let msg = citate[num];
        // применение объекта-обработчика
        btn.addEventListener("click", {handleEvent : loadCitate,
```

```
        msg : msg} );
    })
    // функция инструкция
    function loadCitate(){
        alert(this.msg)
    }
</script>
```

Следующий пример для тех, кто любит покопаться в коде.

example_11. example_11.html

```
<div id="personnel">
    <div data-id="11">Бородина Ксения Алексеевна</div>
    <div data-id="23">Красикова Вера Николаевна</div>
    <div data-id="51">Рыбкина Ольга Витальевна</div>
    <div data-id="44">Маркова Елизавета Андреевна</div>
    <div data-id="75">Ильчук Виталина Георгиевна</div>
</div>
<hr>
<!-- контейнер для вставки данных -->
<div id="console"></div>
<!-- массив данных педагогов -->
<script src="personnels.js"></script>
<script>
    // обойдем в цикле коллекцию элементов div
    document.querySelectorAll("div#personnel div").forEach (
    (div) => {
```



```
// навесим событие click на каждый элемент div
div.addEventListener("click",
// объект-обработчик
{
    // метод handleEvent объекта-обработчика
    handleEvent() {
        // поиск преподавателя по его id
        data = personnels.find( (el) => {
            return this.id == el.id_personnel;
        })
        // console.log(this.person);
        // console.log(data);
        // формирование строки для вывода
        str = `
            <h2>${this.person}</h2>
            <b>Категория:</b> ${data.category}<br>
            <b>Email:</b> ${data.email}<br>
            <b>Опыт работы в колледже:</b>
            ${data.experience_college}<br>
            <b>Общий опыт работы:</b>
            ${data.experience_total}<br>
            <b>Образование:</b> ${data.level_edu}<br>
            <b>Должность:</b> ${data.post}<br>
            <b>Сайт:</b> ${data.site}<br>
            <hr>
        `;
        // вставка контента в документ
```



```
            document.getElementById("console").innerHTML =
            str;
        },
        // свойства объекта-обработчика
        id : div.dataset.id,
        person : div.innerText
    })
});
</script>
```

Фрагмент подключаемого массива обезличенных данных педагогов приведен в листинге **example_11/personnels.js**.

example_11. personnels.js

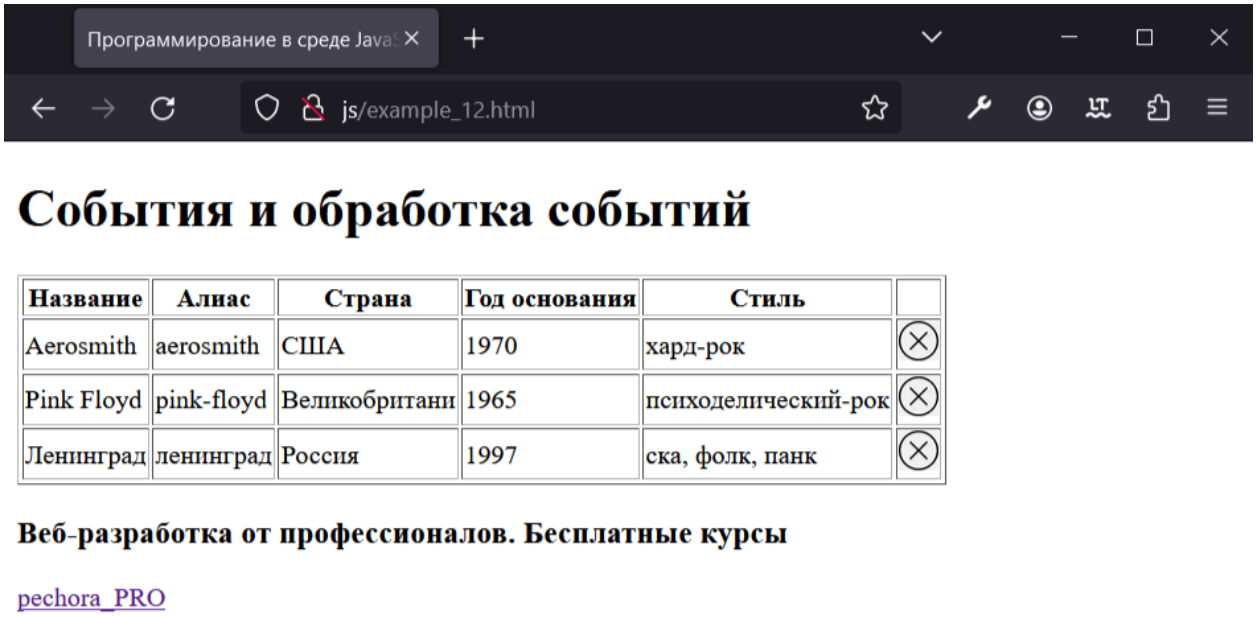
```
// массив обезличенных данных педагогов
let personnels = new Array (
{
    'id_personnel' : 11,
    'post' : 'Преподаватель',
    'category' : 'Соответствие занимаемой должности',
    'level_edu' : 'Высшее профессиональное ',
    'experience_total' : 21.3,
    'experience_college' : 14.5,
    'email' : 'borodina_xall@mail.ru',
    'site' : 'borodina.wordpress.com'
},
    /* ... */
);
```

Базовую часть урока прошли, теперь несколько примеров из практики применения метода **addEventListener**().

Практика применения

Приведу несколько показательных примеров из практики применения метода **addEventListener**().

В примере **example-12** будем удалять строки с альбомами при клике по пиктограмме.



example_12.

```
<table border="1px">
    <tr>
        <th>Название</th>
        <th>Алиас</th>
        <th>Страна</th>
        <th>Год основания</th>
```

```
            <th>Стиль</th>
            <td></td>
        </tr>
        <tr>
            <td>Aerosmith</td>
            <td>aerosmith</td>
            <td>США</td>
            <td>1970</td>
            <td>хард-рок</td>
            <td><img src='icon.jpg' width="24"></td>
        </tr>
        <!-- /// -->
</table>
<script>
    // навесим обработчик на загрузку документа
    document.addEventListener("DOMContentLoaded",
    addHandlerClick)
    // определим функцию, навесим обработчик на click по
    пиктограмме
    function addHandlerClick() {
        // выбор пиктограмм
        let images = document.querySelectorAll("table tr td
        img");
        // перебираем все пиктограммы
        images.forEach ( (img) => {
            // навешиваем обработчики
            img.addEventListener('click', fnClickInput);
```

```
        })
    }
    // функция скрытия строк
    function fnClickInput() {
        let elem = this.closest('tr').style.display = 'none';
    }
</script>
```

В примере мы динамически добавили (говорят – *на лету*) функцию скрытия строки. Все хорошо работает. Пока.

В следующем примере реализуем функцию динамического добавления строки таблицы.

example_13.

```
<h2>Добавь свой вариант</h2>
<form id="frm" onsubmit="return addRow()">
    Название: <input type="text" name="name" value=""><br>
    Алиас: <input type="text" name="alias" value=""><br>
    Страна: <input type="text" name="country" value=""><br>
    Год основания: <input type="text" name="date" value=""><br>
    Стиль: <input type="text" name="style" value=""><br>
    <button>Добавить</button>
</form>
<script>
    // навесим обработчик на загрузку документа
    document.addEventListener("DOMContentLoaded",
    addHandlerClick)
    // определим функцию, навесим обработчик на click по
```

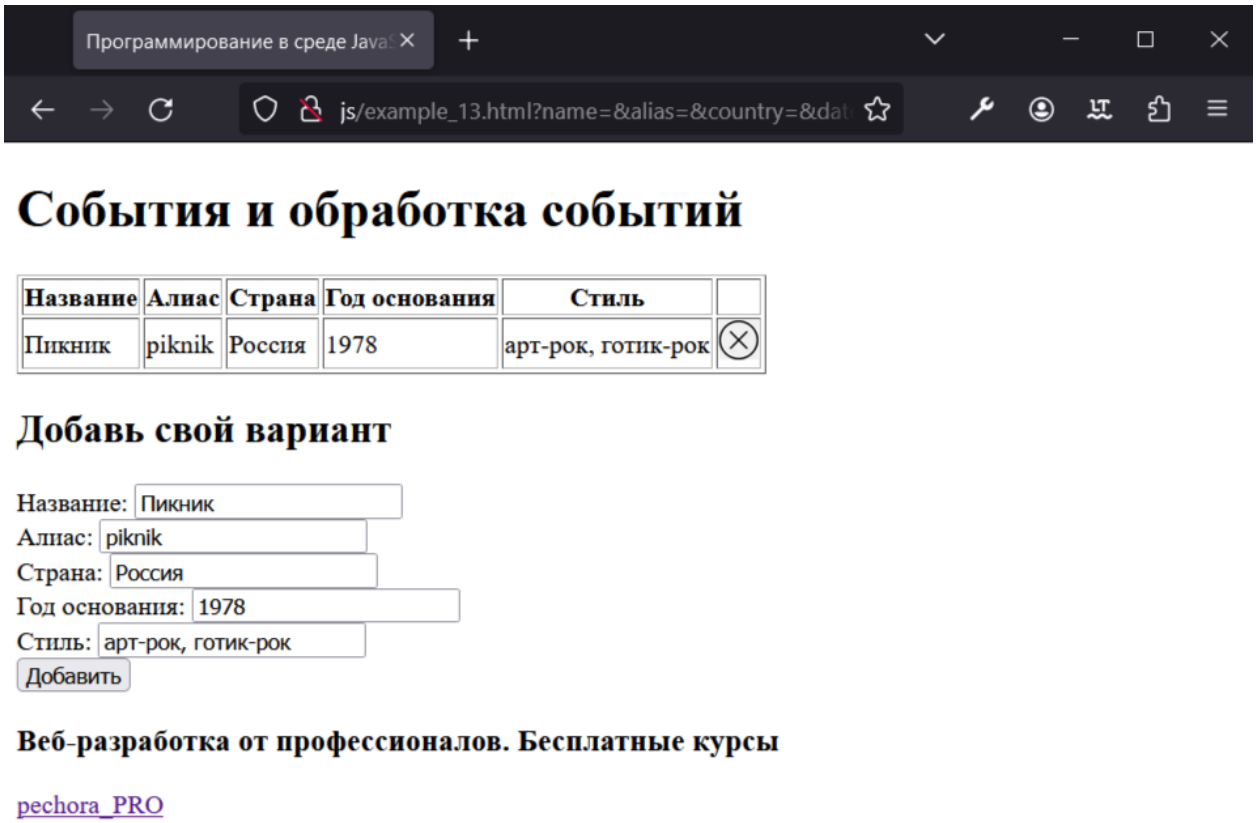
```
пиктограмме
function addHandlerClick() {
    // выбор пиктограмм
    let images = document.querySelectorAll("table tr td
    img");
    // перебираем все пиктограммы
    images.forEach ( (img) => {
        // навешиваем обработчики
        img.addEventListener('click', fnClickInput);
    })
}
// функция скрытия строк
function fnClickInput() {
    let elem = this.closest('tr').style.display = 'none';
}
function addRow() {
    // формируем строку
    let row = `
        <tr>
            <td>${frm.elements['name'].value}</td>
            <td>${frm.elements['alias'].value}</td>
            <td>${frm.elements['country'].value}</td>

            <td>${frm.elements['date'].value}</td>
            <td>${frm.elements['style'].value}</td>
            <td><img src='icon.jpg' width="24"></td>
        </tr>
```

```
            `;
            // вставка контента
            tbl.insertAdjacentHTML("beforeend", row);
            return false;
        }
</script>
```

А вот теперь, **динамически добавленная строка не удаляется**. Все логично, при навешивании обработчика события click по пиктограмме – строки просто **не существовало**.

> ***Важно***. Проблема может иметь несколько решений. Классическое решение – **делегирование событий**, рассмотрим в следующем уроке.



В демонстрационных примерах **example_14** – **example_15** за внешним отличием приложений не пропустите важную особенность – поведение скрипта.

Обратите внимание, в демонстрационном примере **example_14** сбор данных пользовательского выбора в массив происходит **перед переходом** на страницу **target.html** (по-другому будет вести себя скрипт примера example_15).

example_14. example_14.html

```
<h2>Волшебные атрибуты русских сказок</h2>
<div id="codecs">
    <ul>
        <li><label>Ковёр самолёт</label></li>
        <li><label>Шапка невидимка</label></li>
        <!-- /// -->
    </ul>
</div>
<a href="target.html" id="link">Передать данные</a>
<script>
    document.addEventListener("DOMContentLoaded", function() {
        lis = document.querySelectorAll("ul li");
        // вставим элементы checkbox в список
        lis.forEach( li => {
            li.insertAdjacentHTML("beforeend", "<input
            type='checkbox'>");
        })
    })
    // перед переходом на target.html сохраним пользовательский
    выбор
    link.onclick = function savetList() {
        arr = new Array();
        // получаем коллекцию input:checked
```

```
        let inputs = document.querySelectorAll("#codecs ul li
        input:checked");
        // проходим по коллекции
        inputs.forEach ( (input) => {
            // добавляем в массив текст пункта списка
            arr.push(input.previousElementSibling.innerText)
        })
        // сохраняем в localStorage JSON-данные
        window.localStorage.setItem("json", JSON.stringify(arr));
        // alert(JSON.stringify(arr));
    }
</script>
```

example_14. target.html

```
<script>
    // извлекаем JSON-объект из localStorage
    let json = window.localStorage.getItem("json");
    // парсим JSON в JS-объект
    arr = JSON.parse(json);
    document.write('<ol>');
    // вывод в браузер списка
    arr.forEach( elem => {
        document.write(`
            <li>${elem}</li>
        `)
    })
```

```
    document.write('</ol>');
</script>
```

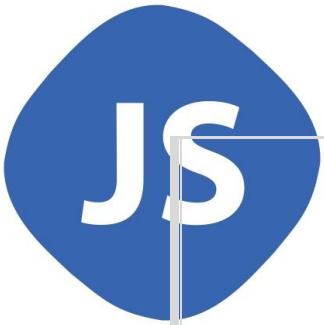
Обратите внимание, в демонстрационном примере **example_15** сбор данных пользовательского выбора в массив происходит **во время события** изменения состояния (change) элемента <input type="checkbox">.

example_15.

```
<table border="1px">
    <tr>
        <td>1</td>
        <td>Atom Heart Mother</td>
        <td>10 октября 1970</td>
        <td>EMI, Harvest, Capitol</td>
        <td>LP, CD</td>
        <td>Золотой (USA)</td>
        <td><input type="checkbox"></td>
    </tr>
    <!-- /// -->
</table>
<p><button id="btn">Вывод выбранных альбомов</button></p>
<!-- контейнер -->
<div id="con"></div>
<script>
    // при построениии DOM сразу навесим обработчики
    document.addEventListener("DOMContentLoaded", loadDOM);
    btn.onclick = show;
    // описание функции определения обработчиков change элементов
```

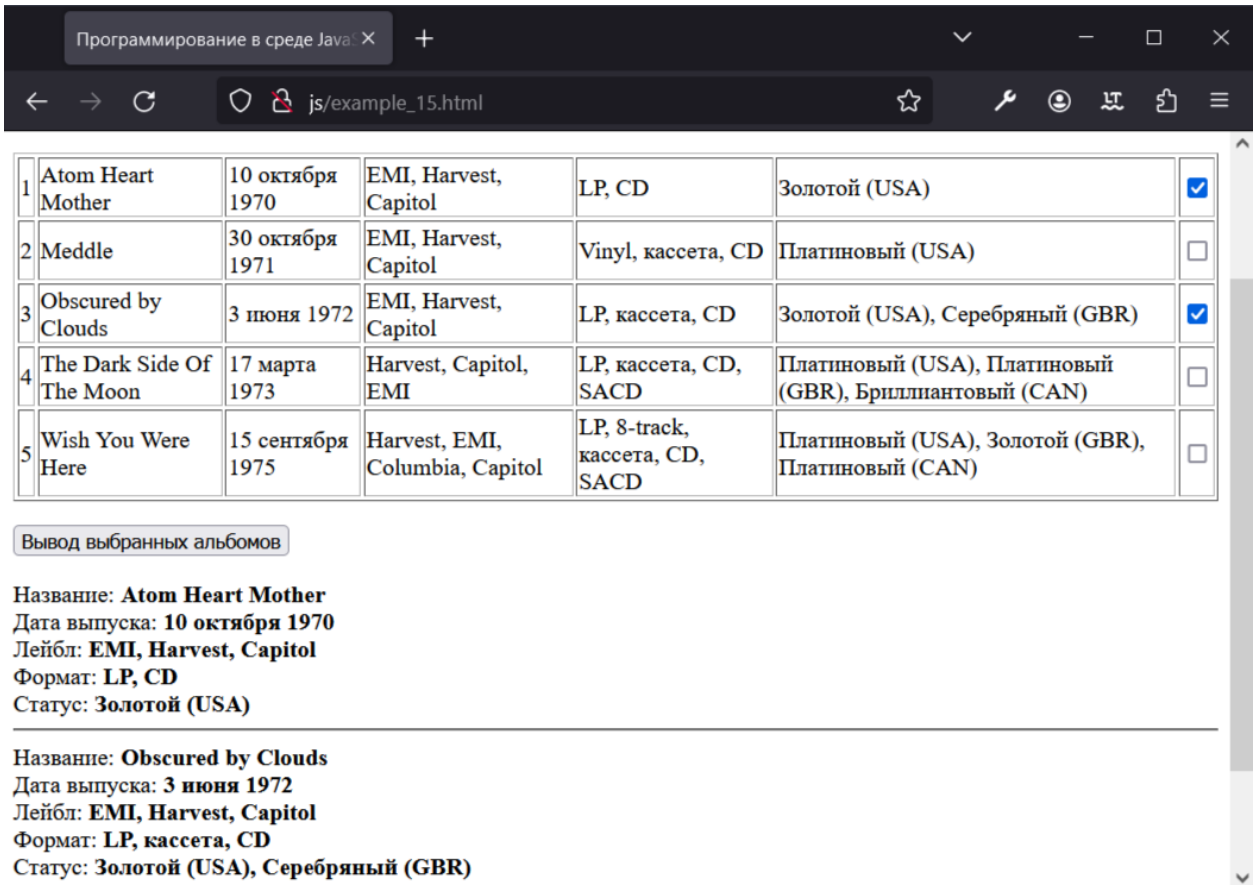
```
input

function loadDOM() {
    // выбор всех элементов input
    let inputs = document.querySelectorAll('table tr input');
    // массив сбора данных
    arr = [];
    // навешивание обработчика на каждый input
    inputs.forEach( (input) => {
        input.addEventListener("change", fnHandle);
    })
}
// функция создания массива отобранных элементов
function fnHandle() {
    tds = this.closest("tr").children;
    // если input выбран
    if (this.checked) {
        // добавляем в массив объект данных
        arr.push (
        {
            id: tds.item(0).innerText,
            name: tds.item(1).innerText,
            date: tds.item(2).innerText,
            label: tds.item(3).innerText,
            format: tds.item(4).innerText,
            status: tds.item(5).innerText
        })
```



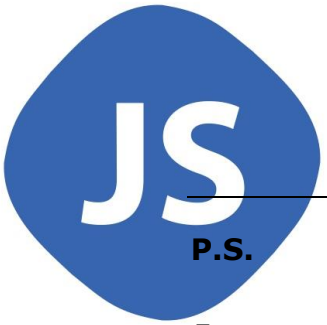
```
            // если input деактивирован
        } else {
            // перезаписываем массив
            // в новый массив не войдет объект деактивированной
            строки
            arr = arr.filter( (obj) => {
                return (tds.item(0).innerText !== obj.id)
            })
        }
        // тестируем содержимое массива
        // console.log(arr)
    }
    // вывод отобранных данных в браузер
    function show() {
        // очишаем контейнер
        con.innerHTML = "";
        // формируем строку для вывода
        arr.forEach( (obj) => {
            out = `
                Название: <b>${obj.name}</b><br>
                Дата выпуска: <b>${obj.date}</b><br>
                Лейбл: <b>${obj.label}</b><br>
                Формат: <b>${obj.format}</b><br>
                Статус: <b>${obj.status}</b><br>
                <hr>
            `
```

```
            // вставка строки в контейнер
            con.insertAdjacentHTML("beforeend", out);
        })
    }
</script>
```



В **самостоятельных работах** к уроку выполним следующие задания:

- Поиск и подсветка объекта в табличном документе.
- Выбор карточек товаров и добавление выбранного в корзину.
- Динамическое добавление элементов в список. Выбор и вывод отмеченных элементов списка.
- Динамическое создание списка из массива. Навешивание на элементы списка события для просмотра пунктов связанного массива.



P.S.

Для отработки и закрепления учебного курса **в расширенной платной версии** предоставляется следующий **раздаточный** материал.

К каждому **уроку** курса:

- Файлы **демонстрационного кода** (example).
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой **теме** курса:

- **Практические работы**.

К **курсу**:

- **Дополнительный практикум.**